AIES- LAB EXPERIMENTS (11-15)

Experiment 11: Crypt Arithmetic Problem

Aim: To solve the cryptarithmetic puzzle, SEND + MORE = MONEY by assigning unique digits to each letter.

CODE:

```
from itertools import permutations

letters = 'SENDMORY'

for p in permutations(range(10), len(letters)):
    d = dict(zip(letters, p))

    if d['S'] == 0 or d['M'] == 0:
        continue

    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = (10000*d['M'] + 1000*d['O'] +
             100*d['N'] + 10*d['E'] + d['Y'])

    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        print("Solution found!")
        break
```

OUTPUT:

```
Enter "help" below
>>>

============== RES
SEND = 9567
MORE = 1085
MONEY = 10652
Solution found!
>>>
```

Result: The cryptarithmetic puzzle was solved by assigning distinct digits to the letters while satisfying the addition of constraint.

Experiment 12: Decision Tree Classifier

Aim: To build a Decision Tree Classifier and predict output based on given input data.

CODE:

```
from sklearn.tree import DecisionTreeClassifier

# Training data
X = [[1, 20], [2, 25], [3, 30], [4, 35],
     [5, 40], [6, 45]]

# Output labels
y = [0, 0, 0, 1, 1, 1]

# Create decision tree model
model = DecisionTreeClassifier()
model.fit(X, y)

# Predict new data
age = int(input("Enter age: "))
salary = int(input("Enter salary: "))

result = model.predict([[age, salary]])

if result[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")
```

OUTPUT:

```
========== RESTART
Enter age: 18
Enter salary: 50000
Loan Approved
>>>
```

Result: The Decision Tree Classifier was implemented successfully to predict loan approval based on input data.

Experiment 13: Artificial Neural Network

Aim: To implement an Artificial Neural Network and train it to predict output using sample data.

CODE:

```
from sklearn.neural_network import MLPClassifier

# Training data
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]

# Create neural network
model = MLPClassifier(hidden_layer_sizes=(4,),
                      activation='relu',
                      max_iter=5000,
                      random_state=1)

# Train the model
model.fit(X, y)

# Test the model
result = model.predict([[1, 0]])

print("Input: [1, 0]")
print("Predicted Output:", result[0])
```

OUTPUT:

```
>>>
    ============== RESTART
    Input: [1, 0]
    Predicted Output: 1
>>>
```

Result: The Artificial Neural Network was trained and tested successfully to predict the output.

Experiment 14: Diet Recommendation Expert System

Aim: To develop a simple expert system that recommends a diet based on the user's age and BMI.

CODE:

```
# Diet Recommendation Expert System

age = int(input("Enter your age: "))
weight = float(input("Enter weight in kg: "))
height = float(input("Enter height in meters: "))

# Calculate BMI
bmi = weight / (height * height)

print("BMI:", round(bmi, 2))

# Diet recommendation
if age < 18:
    print("Recommendation: Balanced growth diet")
elif bmi < 18.5:
    print("Recommendation: Protein and nutrient-rich diet")
elif bmi < 25:
    print("Recommendation: Maintain a balanced diet")
elif bmi < 30:
    print("Recommendation: Healthy diet with portion control")
else:
    print("Recommendation: Consult a dietitian for a suitable diet plan")
```

OUTPUT:

```
========== RESTART: /Users/syedmadeehaanjum/Documents/exp.
Enter your age: 18
Enter weight in kg: 52
Enter height in meters: 154
BMI: 0.0
Recommendation: Protein and nutrient-rich diet
>>>
========== RESTART: /Users/syedmadeehaanjum/Documents/exp.
Enter your age: 18
Enter weight in kg: 78
Enter height in meters: 1.45
BMI: 37.1
Recommendation: Consult a dietitian for a suitable diet plan
>>>
========== RESTART: /Users/syedmadeehaanjum/Documents/exp.
Enter your age: 19
Enter weight in kg: 53
Enter height in meters: 1.54
BMI: 22.35
Recommendation: Maintain a balanced diet
>>>
```

Experiment 15: Family Knowledge Representation

Aim: To represent family relationships using facts and rules in Python.

CODE:

```
family = {
    'Ali': ['Sara', 'Omar'],
    'Sara': ['Zoya', 'Ahmed']
}

for parent, children in family.items():
    for child in children:
        print(parent, "is parent of", child)

name = input("Enter name: ")

if name in family:
    print("Children:", family[name])
else:
    print("No children found")
```

OUTPUT:

